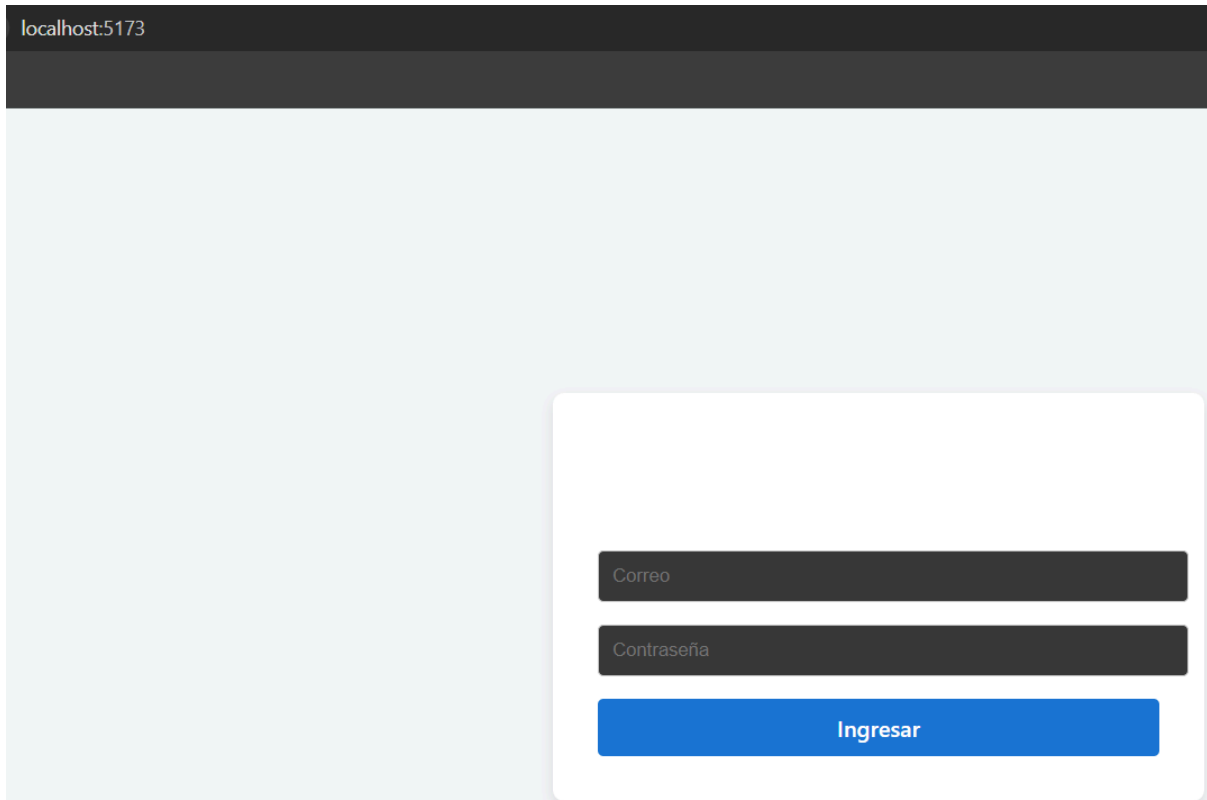


1. Analizar el repositorio base
 - a. Clonar el repositorio Práctica 2.
 - b. Comprender la arquitectura general del sistema:
 - i. Frontend
 - ii. Backend
 - iii. Contenedores
 - c. Verificar que la aplicación es funcional antes de integrar el pipeline

```
PS D:\cursos\Maestria\Modulo12\M12practica12\backend> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
26267e59d217	backend-frontend	"/docker-entrypoint.s..."	17 seconds ago	Up 15 seconds	0.0.0.0:5173->80/tcp, [::]:5173->80/tcp	frontend
0bb5d460bf04	api-gateway:latest	"docker-entrypoint.s..."	17 seconds ago	Up 16 seconds	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	api-gateway
4061414509c4	users-service:latest	"docker-entrypoint.s..."	18 seconds ago	Up 17 seconds	3001/tcp	users-service
c2b36560c6fb	academic-service:latest	"docker-entrypoint.s..."	18 seconds ago	Up 17 seconds	3002/tcp	academic-service



2. Diseñar e implementar el pipeline CI/CD (archivo .yaml) El/la estudiante deberá crear o completar el archivo de pipeline ubicado en .github/workflows/ El pipeline debe incluir, como mínimo, las siguientes etapas:
 - a. Instalación reproducible
 - i. Instalación automática de dependencias.
 - ii. Uso de comandos reproducibles (npm ci, pip install).
 - iii. El pipeline debe fallar si existen inconsistencias.
 - b. Análisis de calidad de código
 - i. Integración de una herramienta de análisis de estilo (ESLint / Pylint).
 - ii. Detección automática de errores comunes y malas prácticas.
 - c. Testing automático
 - i. Ejecución de pruebas unitarias y/o de integración.
 - ii. El pipeline debe detenerse si los tests fallan
 - d. Seguridad del código (SAST)
 - i. Integración de una herramienta de análisis estático de seguridad (Semgrep o SonarQube).

- ii. Detección de vulnerabilidades en el código fuente antes del despliegue.
- e. Seguridad de dependencias (SCA)
 - i. Análisis automático de librerías externas.
 - ii. Detección de vulnerabilidades conocidas (CVEs).
 - iii. El pipeline debe fallar ante riesgos críticos.
- f. Build de contenedores
 - i. Construcción de la imagen Docker del sistema.
 - ii. Versionado del artefacto generado.
- g. Seguridad de contenedores
 - i. Escaneo de la imagen Docker (Trivy o Gype).
 - ii. Detección de vulnerabilidades del sistema base y librerías internas.

3. Justificar las decisiones técnicas (obligatorio)

El/la estudiante deberá elaborar un documento de justificación técnica donde explique:

- Qué herramienta se usa en cada etapa del pipeline.
- En qué fase de DevSecOps actúa.
- Qué tipo de riesgo mitiga.
- Por qué es necesaria incluso cuando el sistema ya es funcional.

Este documento puede entregarse como:

- README.md en el repositorio o
- Documento PDF breve (1–2 páginas).

4. Verificar y evidenciar la ejecución del pipeline

- a. Ejecutar el pipeline CI/CD.
- b. Verificar que todas las etapas se ejecutan correctamente.
- c. Adjuntar evidencia:
 - i. Logs del pipeline
 - ii. Capturas de pantalla
 - iii. Enlaces a ejecuciones exitosas o fallidas justificadas.

Entregables:

Repositorio Git (fork o copia de Práctica 2) que incluya:

- Pipeline CI/CD completo (.yml)
- Código frontend y backend sin modificaciones funcionales mayores
- Documento de justificación técnica
- Evidencia de ejecución del pipeline

devsecops

failed yesterday in 2m 53s

- >  Checkout repository
- >  Setup Node.js
- >  Create env files for Docker Compose
- >  Install & Test users-service
- >  Install & Test academic-service
- >  Install & Test api-gateway
- >  Install & Test frontend
- >  Install Semgrep
- >  SAST user-service
- >  SAST academic-service
- >  SAST api-gateway
- >  Validate env files
- >  Set up Docker Buildx
- >  Build Docker images
- >  Trivy scan users-service
 -  Trivy scan academic-service
 -  Trivy scan api-gateway
 -  Run docker-compose
 -  Smoke test API Gateway
- >  Shutdown services
- >  Post Set up Docker Buildx
 -  Post Setup Node.js
- >  Post Checkout repository
- >  Complete job

✓ Update devsecops workflow para que el paso no falle y emita un warnin... #4

Summary

All jobs



✓ devsecops

Run details

Usage

Workflow file

devsecops

succeeded 3 minutes ago in 3m 41s

- > ✓ Set up job
- > ✓ Build aquasecurity/trivy-action@0.20.0
- > ✓ Checkout repository
- > ✓ Setup Node.js
- > ✓ Create env files for Docker Compose
- > ✓ Install & Test users-service
- > ✓ Install & Test academic-service
- > ✓ Install & Test api-gateway
- > ✓ Install & Test frontend
- > ✓ Install Semgrep
- > ✓ SAST user-service
- > ✓ SAST academic-service
- > ✓ SAST api-gateway
- > ✓ Validate env files
- > ✓ Set up Docker Buildx
- > ✓ Build Docker images
- > ✓ Trivy scan users-service
- > ✓ Trivy scan academic-service
- > ✓ Trivy scan api-gateway
- > ✓ Run docker-compose
- > ✓ Smoke test API Gateway
- > ✓ Shutdown services